

CL Implementation Flow — Inputs, Outputs & Execution Order

Edge Plant/Leaf Classification | MobileNetV3 | PlantVillage

Purpose: This document defines the exact execution order, inputs, outputs, stored artifacts, and graphs required at every stage of the continual learning experiments. Use this as your implementation guide and research journal reference to know what must exist before each stage begins and what must be saved after it completes.

Parallel vs Sequential — Master Execution Map



[DEPLOYMENT] ————— SEQUENTIAL (after all CL done)

Rules

- **PARALLEL** = can run simultaneously on separate GPUs or machines
- **SEQUENTIAL** = must wait for the previous step to fully complete and save outputs
- Never start a CL Cycle 2 before Cycle 1 model weights and parameters are saved
- Never start Part 2 before all 3 base models (M1, M2, M3) are evaluated and saved

PHASE 0 — Data Preparation

Run locally. Must complete fully before any training begins.

Input

00_raw_segmented/ ← From segmentation pipeline (previous stage)
└─ All 35 used class folders

Process

- Leaf-grouped stratified split
- Replay buffer sampling
- Stats logging

Outputs — What to Store

File / Folder	Type	Purpose
data/split_info.json	JSON	Every image path, class, leaf_id, split assignment
data/data_stats.csv	CSV	Image counts per class per split
data/02_tomato_5cls/initial_train/	Images	Part 1 training data
data/02_tomato_5cls/cl_cycle1_stream/	Images	CL Cycle 1 new data
data/02_tomato_5cls/cl_cycle2_stream/	Images	CL Cycle 2 new data

File / Folder	Type	Purpose
<code>data/02_tomato_5cls/val/</code>	Images	Validation — all stages
<code>data/02_tomato_5cls/test/</code>	Images	FIXED test — never touched again
<code>data/02_tomato_5cls/replay_buffer/</code>	Images	EWC Fisher + Replay mixing
<code>data/03_tomato_new_2cls/cl_cycle1_stream/</code>	Images	Cat B Cycle 1 new class data
<code>data/03_tomato_new_2cls/cl_cycle2_stream/</code>	Images	Cat B Cycle 2 new class data
<code>data/03_tomato_new_2cls/val/</code>	Images	New class val
<code>data/03_tomato_new_2cls/test/</code>	Images	FIXED new class test
<code>data/01_pretrain_26cls/train/</code>	Images	M3 Phase A training
<code>data/01_pretrain_26cls/test/</code>	Images	M3 Phase A verification

Graphs to Generate

Graph	File	What it Shows
Class distribution bar chart (5 Tomato)	<code>plots/data/tomato_5cls_distribution.png</code>	Image count per class, color-coded by split
Class distribution bar chart (26 pretrain)	<code>plots/data/pretrain_26cls_distribution.png</code>	Imbalance across 26 classes
Split ratio pie chart (per class)	<code>plots/data/split_ratios.png</code>	Visual of train/val/test/cycle proportions
Class weight table	<code>plots/data/class_weights.png</code>	Computed weights for weighted loss

PHASE 1A — Case 1: Train From Scratch → M1

Inputs Required

```
data/02_tomato_5cls/initial_train/    ← training data
data/02_tomato_5cls/val/              ← validation data
configs/case1_scratch.yaml            ← hyperparameters, class weights
```

Process

- MobileNetV3-Small initialized with random weights
- Classifier head → 5 outputs
- Class-weighted CrossEntropy loss
- Train all layers from scratch
- Early stopping on val accuracy (patience=10, min_epochs=5)

Outputs — What to Store

Artifact	Path	Format	Used In
Best model weights	<code>models/M1_scratch/best_model.pth</code>	PyTorch state dict	Part 2 CL training
Final epoch weights	<code>models/M1_scratch/final_model.pth</code>	PyTorch state dict	Backup
Training config	<code>models/M1_scratch/config.json</code>	JSON	Reproducibility
Training log	<code>logs/M1_scratch_training.csv</code>	CSV (epoch, train_loss, val_loss, val_acc, lr)	Training curve plot
Class weights used	<code>models/M1_scratch/class_weights.json</code>	JSON	Documentation

Graphs to Generate

Graph	File	What it Shows
Training & val loss curve	plots/part1/M1_loss_curve.png	Overfitting detection
Training & val accuracy curve	plots/part1/M1_acc_curve.png	Convergence
Learning rate schedule	plots/part1/M1_lr_schedule.png	LR decay over epochs

📁 PHASE 1B — Case 2: ImageNet Fine-tune → M2

🔑 Inputs Required

```
ImageNet pretrained weights          ← via torchvision (auto-download)
data/02_tomato_5cls/initial_train/
data/02_tomato_5cls/val/
configs/case2_imagenet.yaml
```

Process

- Load MobileNetV3-Small (ImageNet pretrained)
- Freeze backbone → train classifier head only (Phase i)
- Optionally unfreeze last 2 conv blocks at lower LR (Phase ii)
- Class-weighted CrossEntropy loss

➡ Outputs — What to Store

Artifact	Path	Format	Used In
Best model weights	models/M2_imagenet/best_model.pth	PyTorch state dict	Part 2 CL training
Phase i checkpoint	models/M2_imagenet/phase_i_classifier_only.pth	PyTorch state dict	Ablation reference
Training config	models/M2_imagenet/config.json	JSON	Reproducibility

Artifact	Path	Format	Used In
Training log (phase i)	logs/M2_imagenet_phase_i.csv	CSV	Training curve plot
Training log (phase ii)	logs/M2_imagenet_phase_ii.csv	CSV	Training curve plot
Class weights used	models/M2_imagenet/class_weights.json	JSON	Documentation

Graphs to Generate

Graph	File	What it Shows
Loss curve (both phases)	plots/part1/M2_loss_curve.png	Effect of unfreezing
Accuracy curve (both phases)	plots/part1/M2_acc_curve.png	Jump at unfreeze point

 PHASE 1C — Case 3: PlantVillage Pretrain → M3

 **Internal Sequential:** Phase A must complete and save backbone before Phase B begins.

Phase A — Pre-train on 26 Classes

 Inputs Required

```
ImageNet pretrained weights          ← via torchvision
data/01_pretrain_26cls/train/
data/01_pretrain_26cls/test/
configs/case3_phaseA_pretrain.yaml
```

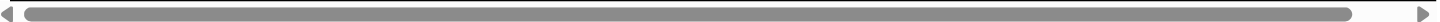
Process

- Load MobileNetV3-Small (ImageNet pretrained)
- Classifier → 26 outputs
- Capped class-weighted loss (max 3×) — critical for 15:1 imbalance

- Train full network

 Outputs — What to Store

Artifact	Path	Format	Used In
Pretrained backbone	<code>models/M3_plantvillage/pretrained_backbone.pth</code>	PyTorch state dict	Phase B — Critical
Feature extractor only	<code>models/M3_plantvillage/backbone_features_only.pth</code>	PyTorch state dict	Transfer analysis
Training config	<code>models/M3_plantvillage/config_phaseA.json</code>	JSON	Reproducibility
Training log	<code>logs/M3_phaseA_training.csv</code>	CSV	Training curve plot
Phase A test results	<code>results/part1/M3_phaseA_test_results.json</code>	JSON	Backbone quality verification
Class weights (26 cls)	<code>models/M3_plantvillage/class_weights_26cls.json</code>	JSON	Documentator



Graphs to Generate

Graph	File	What it Shows
Loss & acc curve (26 cls)	<code>plots/part1/M3_phaseA_curves.png</code>	Backbone training quality
Confusion matrix (26 cls)	<code>plots/part1/M3_phaseA_confusion.png</code>	Which classes backbone learned
Per-class accuracy (26 cls)	<code>plots/part1/M3_phaseA_per_class_acc.png</code>	Imbalance effect check

Phase B — Fine-tune on 5 Tomato Classes

← Inputs Required

```
models/M3_plantvillage/pretrained_backbone.pth      ← FROM PHASE A
data/02_tomato_5cls/initial_train/
data/02_tomato_5cls/val/
configs/case3_phaseB_finetune.yaml
```

Process

- Load backbone from Phase A
- Replace classifier → 5 outputs
- Freeze backbone → train classifier (Phase i)
- Unfreeze last 2 blocks at lower LR (Phase ii)
- Standard class-weighted CrossEntropy (5-class weights)

→ Outputs — What to Store

Artifact	Path	Format	Used In
Best model weights	models/M3_plantvillage/best_model.pth	PyTorch state dict	Part 2 CL training
Training config	models/M3_plantvillage/config_phaseB.json	JSON	Reproducibility
Training log	logs/M3_phaseB_training.csv	CSV	Training curve plot
Class weights (5 cls)	models/M3_plantvillage/class_weights_5cls.json	JSON	Documentation

Graphs to Generate

Graph	File	What it Shows
Loss & acc curve	plots/part1/M3_phaseB_curves.png	Fine-tuning convergence

PHASE 1D — Part 1 Baseline Evaluation (Before Any CL)

| Run for M1, M2, M3 simultaneously (parallel).

Inputs Required

```
models/M1_scratch/best_model.pth
models/M2_imagenet/best_model.pth
models/M3_plantvillage/best_model.pth
data/02_tomato_5cls/test/          ← FIXED test set, first use
```

Outputs — What to Store

Artifact	Path	Format	Used In
Baseline metrics (all 3)	results/part1_baseline/baseline_metrics.csv	CSV (model, acc, macro_f1, per_class_f1)	Paper Table 1, CL forgetting reference
Per-class F1 (M1)	results/part1_baseline/M1_per_class_f1.json	JSON	Forgetting computation
Per-class F1 (M2)	results/part1_baseline/M2_per_class_f1.json	JSON	Forgetting computation
Per-class F1 (M3)	results/part1_baseline/M3_per_class_f1.json	JSON	Forgetting computation
Confusion matrix (M1)	results/part1_baseline/M1_confusion_matrix.npy	NumPy	Plot generation
Confusion matrix (M2)	results/part1_baseline/M2_confusion_matrix.npy	NumPy	Plot generation
Confusion matrix (M3)	results/part1_baseline/M3_confusion_matrix.npy	NumPy	Plot generation

Graphs to Generate

Graph	File	What it Shows
Baseline accuracy bar chart	<code>plots/part1/baseline_accuracy_comparison.png</code>	M1 vs M2 vs M3
Baseline F1 bar chart	<code>plots/part1/baseline_f1_comparison.png</code>	Per-model macro F1
Confusion matrix — M1	<code>plots/part1/M1_baseline_confusion.png</code>	Class-level errors
Confusion matrix — M2	<code>plots/part1/M2_baseline_confusion.png</code>	Class-level errors
Confusion matrix — M3	<code>plots/part1/M3_baseline_confusion.png</code>	Class-level errors
Per-class F1 grouped bar	<code>plots/part1/baseline_per_class_f1.png</code>	All 3 models × 5 classes

 PHASE 2A — Category A: Same Classes CL

For each method below: run M1, M2, M3 in parallel. Cycle 1 → Cycle 2 is sequential per method.

A0 — Baseline Record (Before CL Update)

Simply record the Part 1 baseline results as the A0 reference point. No new computation needed — pull from `results/part1_baseline/`.

Method A1 — EWC

CYCLE 1

 Inputs Required

models/{M1,M2,M3}/best_model.pth	← Base models from Part 1
data/02_tomato_5cls/replay_buffer/	← For Fisher computation
data/02_tomato_5cls/cl_cycle1_stream/	← New training data
data/02_tomato_5cls/val/	← Early stopping (monitors forgetting)
data/02_tomato_5cls/test/	← Evaluation
configs/ewc_cycle1.yaml	← λ , LR, epochs

Process

1. Load base model
2. Compute Fisher Information Matrix using `replay_buffer/`
3. Store θ^* (current weights before update)
4. Train on `cl_cycle1_stream/` with EWC loss: $L = \text{CrossEntropy}(\text{new_data}) + \lambda \times \sum F_i(\theta_i - \theta^*_i)^2$
5. Early stop when val accuracy on old classes drops

→ Outputs — What to Store

Artifact	Path	Format	Used In
Updated model	<code>results/cat_a/ewc/{M}/cycle1/model.pth</code>	PyTorch state dict	Cycle 2 input
Fisher matrix	<code>results/cat_a/ewc/{M}/cycle1/fisher_matrix.pkl</code>	Pickle	Cycle 2 EWC penalty
Optimal params θ^*	<code>results/cat_a/ewc/{M}/cycle1/optimal_params.pkl</code>	Pickle	Cycle 2 EWC penalty
Metrics	<code>results/cat_a/ewc/{M}/cycle1/metrics.json</code>	JSON	Summary table
Training log	<code>logs/cat_a/ewc_{M}_cycle1.csv</code>	CSV	Training curve
λ value used	<code>results/cat_a/ewc/{M}/cycle1/config.json</code>	JSON	Reproducibility

CYCLE 2

← Inputs Required

```
results/cat_a/ewc/{M}/cycle1/model.pth           ← FROM CYCLE 1
results/cat_a/ewc/{M}/cycle1/fisher_matrix.pkl    ← FROM CYCLE 1 ← CRITICAL
results/cat_a/ewc/{M}/cycle1/optimal_params.pkl   ← FROM CYCLE 1 ← CRITICAL
data/02_tomato_5cls/cl_cycle2_stream/
data/02_tomato_5cls/val/
data/02_tomato_5cls/test/
```

→ Outputs — What to Store

Artifact	Path	Format	Used In
Final model	results/cat_a/ewc/{M}/cycle2/model.pth	PyTorch state dict	Deployment candidate
Updated Fisher	results/cat_a/ewc/{M}/cycle2/fisher_matrix.pkl	Pickle	Future cycles / documentation
Updated θ^*	results/cat_a/ewc/{M}/cycle2/optimal_params.pkl	Pickle	Documentation
Metrics	results/cat_a/ewc/{M}/cycle2/metrics.json	JSON	Summary table
Training log	logs/cat_a/ewc_{M}_cycle2.csv	CSV	Training curve

Method A2 — Experience Replay

CYCLE 1

← Inputs Required

```
models/{M1,M2,M3}/best_model.pth
data/02_tomato_5cls/replay_buffer/           ← Mixed into training batches
data/02_tomato_5cls/cl_cycle1_stream/        ← New data
data/02_tomato_5cls/val/
data/02_tomato_5cls/test/
configs/replay_cycle1.yaml
```

Process

- 1. Load base model
- 2. Each training batch = samples from (cl_cycle1_stream/) + samples from (replay_buffer/)
- 3. Stratified sampling — equal class representation in buffer samples per batch
- 4. Class-weighted CrossEntropy on mixed batches
- 5. Early stop monitoring val accuracy on old classes
- 6. After training: update replay buffer — add portion of (cl_cycle1_stream/) samples

➡ Outputs — What to Store

Artifact	Path	Format	Use
Updated model	results/cat_a/replay/{M}/cycle1/model.pth	PyTorch state dict	Cycle input
Updated replay buffer manifest	results/cat_a/replay/{M}/cycle1/replay_buffer_manifest.json	JSON	Cycle replay samples
Buffer image list	results/cat_a/replay/{M}/cycle1/buffer_image_paths.txt	Text	Accuracy training
Metrics	results/cat_a/replay/{M}/cycle1/metrics.json	JSON	Summary table
Training log	logs/cat_a/replay_{M}_cycle1.csv	CSV	Training accuracy

CYCLE 2

⬅ Inputs Required

```
results/cat_a/replay/{M}/cycle1/model.pth           ← FROM CYCLE 1
results/cat_a/replay/{M}/cycle1/replay_buffer_manifest.json ← FROM CYCLE 1 ←
CRITICAL
data/02_tomato_5cls/cl_cycle2_stream/
data/02_tomato_5cls/val/
data/02_tomato_5cls/test/
```

➡ Outputs — What to Store

Artifact	Path	Format	Use
Final model	results/cat_a/replay/{M}/cycle2/model.pth	PyTorch state dict	Deployment
Final buffer manifest	results/cat_a/replay/{M}/cycle2/replay_buffer_manifest.json	JSON	Documentation
Metrics	results/cat_a/replay/{M}/cycle2/metrics.json	JSON	Summary
Training log	logs/cat_a/replay_{M}_cycle2.csv	CSV	Training

Method A3 — Parameter Isolation

CYCLE 1

⬅ Inputs Required

```
models/{M1,M2,M3}/best_model.pth
data/02_tomato_5cls/cl_cycle1_stream/
data/02_tomato_5cls/val/
data/02_tomato_5cls/test/
configs/isolation_cycle1.yaml
```

Process

- 1. Load base model — freeze all existing parameters completely
- 2. Add lightweight adapter layers (new trainable parameters only)

- 3. Train only new parameters on (cl_cycle1_stream/
- 4. At inference: route through base + adapter combined

➔ Outputs — What to Store

Artifact	Path	Format	Used In
Expanded model	results/cat_a/isolation/{M}/cycle1/model_expanded.pth	PyTorch state dict	Cycle 2 input
Frozen mask	results/cat_a/isolation/{M}/cycle1/frozen_mask.pkl	Pickle	Cycle 2 — know which params are old
Model size	results/cat_a/isolation/{M}/cycle1/model_size_mb.txt	Text	Deployment comparison
Parameter count	results/cat_a/isolation/{M}/cycle1/param_count.json	JSON	Growth tracking
Metrics	results/cat_a/isolation/{M}/cycle1/metrics.json	JSON	Summary table
Training log	logs/cat_a/isolation_{M}_cycle1.csv	CSV	Training curve

CYCLE 2

⬅ Inputs Required

results/cat_a/isolation/{M}/cycle1/model_expanded.pth ← FROM CYCLE 1

results/cat_a/isolation/{M}/cycle1/frozen_mask.pkl ← FROM CYCLE 1 ← CRITICAL

data/02_tomato_5cls/cl_cycle2_stream/

data/02_tomato_5cls/val/

data/02_tomato_5cls/test/

➔ Outputs — What to Store

Artifact	Path	Format	Used In
Final expanded model	<code>results/cat_a/isolation/{M}/cycle2/model_expanded.pth</code>	PyTorch state dict	Deploymer candidate
Final frozen mask	<code>results/cat_a/isolation/{M}/cycle2/frozen_mask.pkl</code>	Pickle	Documenta
Model size	<code>results/cat_a/isolation/{M}/cycle2/model_size_mb.txt</code>	Text	Growth comparison
Metrics	<code>results/cat_a/isolation/{M}/cycle2/metrics.json</code>	JSON	Summary t

Method A_Base — Naive Fine-tuning

CYCLE 1 & 2

← Inputs Required

```

Cycle 1: models/{M1,M2,M3}/best_model.pth
Cycle 2: results/cat_a/naive/{M}/cycle1/model.pth
data/02_tomato_5cls/cl_cycle{1,2}_stream/    ← New data only, no replay
data/02_tomato_5cls/val/
data/02_tomato_5cls/test/

```

→ Outputs — What to Store

Artifact	Path	Format	Used In
Model (each cycle)	<code>results/cat_a/naive/{M}/cycle{n}/model.pth</code>	PyTorch state dict	Forgetting baseline reference
Metrics	<code>results/cat_a/naive/{M}/cycle{n}/metrics.json</code>	JSON	Summary table — lower bound

Category A — After-Update Evaluation & Summary

➔ Outputs — What to Store

Artifact	Path	Format	Used In
Master summary table	results/cat_a/summary_table.csv	CSV	Paper Table 2
Forgetting scores	results/cat_a/forgetting_scores.csv	CSV	Paper forgetting analysis
Per-class F1 all methods	results/cat_a/per_class_f1_all.csv	CSV	Paper per-class analysis

Graphs to Generate

Graph	File	What it Shows
Accuracy vs Cycle (per method, per model)	plots/cat_a/acc_vs_cycle_{M}.png	Stability across cycles
Macro F1 vs Cycle	plots/cat_a/f1_vs_cycle_{M}.png	F1 degradation
Forgetting score bar chart	plots/cat_a/forgetting_scores.png	Method comparison
Backward transfer heatmap	plots/cat_a/backward_transfer_heatmap.png	Models × Methods
Training loss curves (all methods)	plots/cat_a/training_curves_{M}_cycle{n}.png	Convergence per method
Per-class F1 heatmap	plots/cat_a/per_class_f1_heatmap.png	Which classes are forgotten most

NEW PHASE 2B — Category B: New Class Introduced (5 → 7 Classes)

B0 — Baseline Test (Before New Classes Seen)

← Inputs Required

models/{M1,M2,M3}/best_model.pth

data/02_tomato_5cls/test/

data/03_tomato_new_2cls/test/ ← Model has never seen these

→ Outputs — What to Store

Artifact	Path	Format	Used In
B0 metrics (old classes)	results/cat_b/b0_old_class_metrics.json	JSON	Forgetting reference
B0 metrics (new classes)	results/cat_b/b0_new_class_metrics.json	JSON	Near-zero reference — forward transfer baseline

Method B1 — EWC + Head Expansion

CYCLE 1

← Inputs Required

models/{M1,M2,M3}/best_model.pth

data/02_tomato_5cls/replay_buffer/ ← Fisher computation (old classes)

data/02_tomato_5cls/c1_cycle1_stream/ ← Old class new stream

data/03_tomato_new_2cls/c1_cycle1_stream/ ← New classes T6, T7

data/02_tomato_5cls/val/

data/03_tomato_new_2cls/val/

data/02_tomato_5cls/test/

data/03_tomato_new_2cls/test/

Process

1. Compute Fisher on replay_buffer/ with current 5-class head
2. Store θ^* for old 5-class weights
3. Expand classifier: $5 \rightarrow 7$ outputs (new neurons for T6, T7 initialized randomly)
4. Train on combined stream (old + new class data) with EWC penalty on old weights

5. Early stop monitoring val accuracy on old 5 classes

➔ Outputs — What to Store

Artifact	Path	Format	Used In
Updated model (7-class)	results/cat_b/ewc/{M}/cycle1/model.pth	PyTorch state dict	Cycle 2
Fisher matrix (old weights)	results/cat_b/ewc/{M}/cycle1/fisher_matrix.pkl	Pickle	Cycle 2
Optimal params θ^*	results/cat_b/ewc/{M}/cycle1/optimal_params.pkl	Pickle	Cycle 2
Metrics — old classes	results/cat_b/ewc/{M}/cycle1/metrics_old.json	JSON	Forgetting measurement
Metrics — new classes	results/cat_b/ewc/{M}/cycle1/metrics_new.json	JSON	Acquisition measurement
Training log	logs/cat_b/ewc_{M}_cycle1.csv	CSV	Training curve

CYCLE 2

⬅ Inputs Required

results/cat_b/ewc/{M}/cycle1/model.pth

← FROM CYCLE 1

results/cat_b/ewc/{M}/cycle1/fisher_matrix.pkl

← FROM CYCLE 1 ← CRITICAL

results/cat_b/ewc/{M}/cycle1/optimal_params.pkl

← FROM CYCLE 1 ← CRITICAL

data/02_tomato_5cls/cl_cycle2_stream/

data/03_tomato_new_2cls/cl_cycle2_stream/

data/02_tomato_5cls/val/

data/03_tomato_new_2cls/val/

data/02_tomato_5cls/test/

data/03_tomato_new_2cls/test/

➔ **Outputs — What to Store**

Artifact	Path	Format	Used In
Final model (7-class)	results/cat_b/ewc/{M}/cycle2/model.pth	PyTorch state dict	Deployment candidate
Updated Fisher	results/cat_b/ewc/{M}/cycle2/fisher_matrix.pkl	Pickle	Documentation
Metrics — old classes	results/cat_b/ewc/{M}/cycle2/metrics_old.json	JSON	Summary table
Metrics — new classes	results/cat_b/ewc/{M}/cycle2/metrics_new.json	JSON	Summary table

Method B2 — Experience Replay + New Classes

CYCLE 1

⬅ **Inputs Required**

```
models/{M1,M2,M3}/best_model.pth
data/02_tomato_5cls/replay_buffer/
data/02_tomato_5cls/cl_cycle1_stream/
data/03_tomato_new_2cls/cl_cycle1_stream/
data/02_tomato_5cls/val/ + data/03_tomato_new_2cls/val/
data/02_tomato_5cls/test/ + data/03_tomato_new_2cls/test/
```

Process

- 1. Expand classifier: 5 → 7 outputs
- 2. Each batch = replay_buffer/ (old classes) + cl_cycle1_stream/ + cl_cycle2_stream/ (new classes)
- 3. Update replay buffer after training — include T6, T7 samples

➔ **Outputs — What to Store**

Artifact	Path	Format	Use
Updated model (7-class)	results/cat_b/replay/{M}/cycle1/model.pth	PyTorch state dict	Cyc
Updated replay buffer manifest	results/cat_b/replay/{M}/cycle1/replay_buffer_manifest.json	JSON	Cyc nov incl T6,
Metrics — old classes	results/cat_b/replay/{M}/cycle1/metrics_old.json	JSON	Sun tabl
Metrics — new classes	results/cat_b/replay/{M}/cycle1/metrics_new.json	JSON	Sun tabl

CYCLE 2

Same pattern as Cycle 1 but uses updated model and buffer from Cycle 1. Save to `cycle2/` folder.

Method B3 — Parameter Isolation (Zero Forgetting Branch)

CYCLE 1

← Inputs Required

```
models/{M1,M2,M3}/best_model.pth      ← Frozen entirely
data/03_tomato_new_2cls/cl_cycle1_stream/ ← Only new class data needed
data/03_tomato_new_2cls/val/
data/02_tomato_5cls/test/ + data/03_tomato_new_2cls/test/
```

Process

1. Freeze entire base model (5-class)
2. Build new lightweight branch for T6, T7 only
3. Train new branch on new class data only
4. At inference: base model handles T1-T5, new branch handles T6-T7

➔ Outputs — What to Store

Artifact	Path	Format	Used In
Frozen base model	results/cat_b/isolation/{M}/cycle1/base_frozen.pth	PyTorch state dict	Cycle 2
New branch weights	results/cat_b/isolation/{M}/cycle1/new_branch.pth	PyTorch state dict	Cycle 2
Combined model size	results/cat_b/isolation/{M}/cycle1/model_size_mb.txt	Text	Deployment comparison
Metrics — old classes	results/cat_b/isolation/{M}/cycle1/metrics_old.json	JSON	Verify zero forgetting
Metrics — new classes	results/cat_b/isolation/{M}/cycle1/metrics_new.json	JSON	New class acquisition

CYCLE 2

← Inputs Required

results/cat_b/isolation/{M}/cycle1/base_frozen.pth ← FROM CYCLE 1

results/cat_b/isolation/{M}/cycle1/new_branch.pth ← FROM CYCLE 1 ← CRITICAL

data/03_tomato_new_2cls/cl_cycle2_stream/

Save to `cycle2/` folder with same structure.

Category B — After-Update Evaluation & Summary

➔ Outputs — What to Store

Artifact	Path	Format	Used In
Master summary table	results/cat_b/summary_table.csv	CSV	Paper Table 3
Forgetting scores (old classes)	results/cat_b/forgetting_scores.csv	CSV	Paper
Forward transfer scores (new)	results/cat_b/forward_transfer_scores.csv	CSV	Paper

Graphs to Generate

Graph	File	What it Shows
Old class acc vs Cycle (all methods)	plots/cat_b/old_acc_vs_cycle_{M}.png	Retention of original knowledge
New class acc vs Cycle (all methods)	plots/cat_b/new_acc_vs_cycle_{M}.png	New class acquisition speed
Plasticity-Stability tradeoff scatter	plots/cat_b/plasticity_stability_{M}.png	Core CL tradeoff visualization
Forgetting vs Forward Transfer	plots/cat_b/forgetting_vs_ft.png	Method comparison across models
Model size growth chart	plots/cat_b/model_size_growth.png	Isolation method size penalty
Per-class F1 before/after (7 classes)	plots/cat_b/per_class_f1_before_after_{M}.png	Full class-level view



PHASE 3 — Edge Deployment

Run after all CL experiments complete. Select best model per category based on results.

- Best backward transfer (least forgetting) for Category A
- Best balance of old retention + new acquisition for Category B
- Smallest model with acceptable accuracy drop

← Inputs Required

```
Selected best CL model weights (.pth)
data/02_tomato_5cls/test/
data/03_tomato_new_2cls/test/
```

Process

1. Structured pruning (20–40% target sparsity)
2. Post-training quantization (INT8 / FP16)
3. Export to ONNX
4. Benchmark inference on CPU / edge device

→ Outputs — What to Store

Artifact	Path	Format	Used In
Pruned model	deployment/pruned/{model_name}.pth	PyTorch state dict	Quantization input
Quantized model	deployment/quantized/{model_name}.onnx	ONNX (INT8)	Edge deployment
Edge benchmark results	deployment/edge_benchmark.csv	CSV	Paper deployment section
Accuracy vs size report	deployment/accuracy_vs_size.json	JSON	Paper Table 4

Graphs to Generate

Graph	File	What it Shows
Accuracy vs Model Size	plots/deployment/acc_vs_size.png	Compression tradeoff

Graph	File	What it Shows
scatter		
Inference time comparison	<code>plots/deployment/inference_time.png</code>	Speed gain from quantization
Pruning sparsity vs accuracy	<code>plots/deployment/pruning_curve.png</code>	How much can be pruned safely

PHASE 4 — Final Results Compilation

Final Outputs for Paper

Artifact	Path	Format	What it is
Master results table	<code>results/final_comparison_table.csv</code>	CSV	All models × methods × cycles × metrics
Part 1 comparison	<code>results/paper_table1_part1.csv</code>	CSV	M1 vs M2 vs M3 baseline
Category A comparison	<code>results/paper_table2_catA.csv</code>	CSV	CL method comparison, same classes
Category B comparison	<code>results/paper_table3_catB.csv</code>	CSV	CL method comparison, new classes
Deployment comparison	<code>results/paper_table4_deployment.csv</code>	CSV	Edge compression results

Final Graphs for Paper

Graph	File	What it Shows
Overall architecture diagram	<code>plots/final/experiment_overview.png</code>	Full pipeline visual
Part 1 baseline comparison	<code>plots/final/part1_comparison.png</code>	M1 vs M2 vs M3

Graph	File	What it Shows
Cat A: Forgetting comparison	plots/final/catA_forgetting.png	EWC vs Replay vs Isolation vs Naive
Cat B: Plasticity-Stability	plots/final/catB_tradeoff.png	Core CL contribution figure
Deployment accuracy-size	plots/final/deployment.png	Edge suitability

 Complete Output Folder Structure Reference

```
cl_research/
|
|─ models/                                ← Part 1 base models
|   |─ M1_scratch/
|   |─ M2_imagenet/
|   └─ M3_plantvillage/
|
|─ results/
|   |─ part1_baseline/                    ← Before CL evaluation
|   |─ cat_a/
|   |   |─ ewc/{M1,M2,M3}/cycle{1,2}/
|   |   |─ replay/{M1,M2,M3}/cycle{1,2}/
|   |   |─ isolation/{M1,M2,M3}/cycle{1,2}/
|   |   |─ naive/{M1,M2,M3}/cycle{1,2}/
|   |   └─ summary_table.csv
|   |─ cat_b/
|   |   |─ ewc/{M1,M2,M3}/cycle{1,2}/
|   |   |─ replay/{M1,M2,M3}/cycle{1,2}/
|   |   |─ isolation/{M1,M2,M3}/cycle{1,2}/
|   |   |─ naive/{M1,M2,M3}/cycle{1,2}/
|   |   └─ summary_table.csv
|   |─ final_comparison_table.csv
|   └─ paper_table{1,2,3,4}.csv
|
|─ logs/                                ← Training logs per experiment
|
|─ plots/
|   |─ data/
|   └─ part1/
```

```

|   |─ cat_a/
|   |─ cat_b/
|   |─ deployment/
|   |─ final/
|
|─ deployment/
|   |─ pruned/
|   |─ quantized/

```

⚠ Critical Cross-Stage Dependencies

If this is missing...	This stage cannot start
<code>models/M3_plantvillage/pretrained_backbone.pth</code>	Case 3 Phase B fine-tuning
<code>models/{M}/best_model.pth</code> (all 3)	Any Part 2 CL training
<code>results/part1_baseline/</code>	CL forgetting score computation
<code>results/cat_a/ewc/{M}/cycle1/fisher_matrix.pkl</code>	EWC Cycle 2
<code>results/cat_a/ewc/{M}/cycle1/optimal_params.pkl</code>	EWC Cycle 2
<code>results/cat_a/replay/{M}/cycle1/model.pth</code>	Replay Cycle 2
<code>results/cat_a/replay/{M}/cycle1/replay_buffer_manifest.json</code>	Replay Cycle 2
<code>results/cat_a/isolation/{M}/cycle1/frozen_mask.pkl</code>	Isolation Cycle 2
<code>results/cat_b/isolation/{M}/cycle1/new_branch.pth</code>	Cat B Isolation Cycle 2

This document covers the full implementation flow from data preparation through edge deployment. Cross-reference with `DATA_README.md` for data folder details and `CL_Research_Journal.md` for theoretical background and evaluation metrics.